

src\main.cpp

```
1  /*
2  This program uses an ESP32-NodeMCU board a 2.4" TFT LCD Shield with integrated Micro-SD
  reader. It uses analoge capacitive soilmoisture sensors that
3  return analog signals. In addition it controls watering by an electronic pump and
  controls the level by a waterlevel sensor.
4
5  It setups a WIFI Server and controls the watering.
6
7  */
8  /*
9   Rui Santos
10   Complete project details at https://RandomNerdTutorials.com/esp32-web-server-microsd-card/
11
12   Permission is hereby granted, free of charge, to any person obtaining a copy
13   of this software and associated documentation files.
14
15   The above copyright notice and this permission notice shall be included in all
16   copies or substantial portions of the Software.
17  */
18
19  #include <Arduino.h>
20  #include <WiFi.h>
21  #include <ctime>
22  #include "time.h"
23  #include <AsyncTCP.h>
24  #include <ESPAsyncWebServer.h>
25  #include "FS.h"
26  #include "SD.h"
27  #include "SPI.h"
28  #include <Adafruit_BMP085_U.h>
29
30  // local header files
31  #include "PinDefinitions.h"
32  #include "WifiSpecs.h"
33
34  #ifndef SSID
35  #define SSID "Replace with your SSID"
36  #endif
37
38  #ifndef PASSWORD
39  #define PASSWORD "Replace with your Password"
40  #endif
41
42
43  const char* ntpServer = "pool.ntp.org";
44  const long  gmtoffset_sec = 3600;
45  const int   daylightOffset_sec = 3600;
46
47  const int wetValue = 1000; // returnValue of a wet Sensor
48  const int dryValue = 4100; // returnValue of a dry Sensor
49
```

```

50 // watering
51 bool watering = false; // wheater watering is nessesary
52 const int wateringValue = 20;
53
54 // name functions
55 int getAverageAnalogPinValue(int GPIO_PIN, int Measurements);
56 float getDistance(void);
57
58 // class of Moisture Sensors
59 class MoistureSensor {
60     int GPIO_PIN;
61
62     public:
63
64     void initialize(int GPIO_PINinput) {
65         GPIO_PIN = GPIO_PINinput;
66         pinMode(GPIO_PIN, INPUT);
67     }
68
69     int getMoistureSensorData() { //Moisture Sensor Data in percent
70
71         int percent = (dryValue - getAverageAnalogPinValue(GPIO_PIN, 10)) *100 / (dryValue -
72         wetValue);
73         percent = constrain(percent, 0, 100);
74
75         return percent; //Function that returns the percentual moisture as an int in Range of
76         0 to 100
77     }
78 };
79
80 MoistureSensor sensors[6]; //crate 6 sensors
81 AsyncWebServer server(80);
82 Adafruit_BMP085_Unified bmp = Adafruit_BMP085_Unified(10085); // classify barometric
83 sensor
84 sensors_event_t event; // structure the sensor data compiling library
85
86 class CSV_Logger {
87     // String of every line
88     char lineBuffer[255];
89     int offset = 0; //offset Position in char array
90
91     public:
92
93     void initSDCard(int CsPinInput){
94         // begin SD Card
95         if(!SD.begin(CsPinInput)){ // Error handling
96             Serial.println("Card Mount Failed");
97             return;
98         }
99         uint8_t cardType = SD.cardType();
100
101         // Card type
102         if(cardType == CARD_NONE){ // Error handling
103             Serial.println("No SD card attached");

```

```
101     return;
102 }
103
104 Serial.print("SD Card Type: ");
105 if(cardType == CARD_MMC){
106     Serial.println("MMC");
107 } else if(cardType == CARD_SD){
108     Serial.println("SDSC");
109 } else if(cardType == CARD_SDHC){
110     Serial.println("SDHC");
111 } else {
112     Serial.println("UNKNOWN");
113 }
114
115 // Card Size
116 uint64_t cardSize = SD.cardSize() / (1024 * 1024);
117 Serial.printf("SD Card Size: %lluMB\n", cardSize);
118 }
119
120 void sendBuffer(fs::FS &fs, const char * path) {
121     File data = fs.open(path, FILE_APPEND);
122     if (!data) return;
123     data.print(lineBuffer);
124     data.close();
125 }
126
127 void addTimeStamp() {
128     time_t now;
129     struct tm timeinfo;
130
131     time(&now);
132     localtime_r(&now, &timeinfo);
133     offset += strftime(lineBuffer + offset, sizeof(lineBuffer)- offset, "%Y-%m-%d
%H:%M:%S;", &timeinfo);
134 }
135
136 void addMoistureData() {
137     int wetSensorData = 0; // Number of wet sensors
138
139     for (int sensor = 0; sensor < 6; sensor++){
140         int moistureData = sensors[sensor].getMoistureSensorData();
141
142         if (moistureData <= wateringValue) { // if any sensor is too dry
143             watering = true;
144         } else {
145             wetSensorData++;
146         }
147         offset += snprintf(lineBuffer + offset, sizeof(lineBuffer) - offset, "%d;",
moistureData);
148     }
149
150     if (wetSensorData == 6) {watering = false;} // deactivate watering if every sensor is
wet
151 }
```

```
152
153 void addWaterLevel() {
154     float distance = 0; // Waterlevel
155     int counter = 0; // Counter for attempts to measure water depth
156
157     do {
158         distance = getDistance(); // as long as the value is not valid
159         counter++;
160     } while(distance == -1 && counter<10); // try 10 times
161
162     offset += snprintf(lineBuffer + offset, sizeof(lineBuffer) - offset, "%.2f;",
distance);
163
164 }
165
166 void addWateringStatus() {
167     offset += snprintf(lineBuffer + offset, sizeof(lineBuffer) - offset, "%d;", watering);
168 }
169
170 void addTemperature() {
171     float temperature;
172     bmp.getTemperature(&temperature);
173
174     offset += snprintf(lineBuffer + offset, sizeof(lineBuffer) - offset, "%.2f;",
temperature);
175 }
176
177 void addPreasure() {
178     float preasure;
179     preasure = event.pressure;
180
181     offset += snprintf(lineBuffer + offset, sizeof(lineBuffer) - offset, "%.2f;",
preasure);
182 }
183 void endLine() {
184     snprintf(lineBuffer + offset, sizeof(lineBuffer)- offset, "\n");
185     offset = 0;
186 }
187
188 void clearBuffer() {
189     memset(lineBuffer, 0, sizeof(lineBuffer));
190     offset = 0;
191 }
192
193 void appendSensorData(void) {
194     clearBuffer();
195     addTimeStamp();
196     addMoistureData();
197     addWaterLevel();
198     addWateringStatus();
199     endLine();
200     sendBuffer(SD, "/data.csv");
201 }
202 };
```

```
203
204 CSV_Logger CSV;
205
206
207 void initWiFi() {
208     WiFi.mode(WIFI_STA);
209     WiFi.begin(SSID, PASSWORD);
210
211     Serial.print("Connecting to WiFi ..");
212
213     while (WiFi.status() != WL_CONNECTED) {
214         Serial.print('.');
215         delay(1000);
216     }
217     Serial.println(WiFi.localIP());
218 }
219
220 int getAverageAnalogPinValue(int GPIO_PIN, int Measurements) { //PinNumber as GPIO Pin
221
222     //int values[10];
223     int value;
224     long sum = 0;
225
226
227     for(int i = 0; i < Measurements; i++){
228         value = analogRead(GPIO_PIN);
229         sum += value;
230         //values[i] = v;
231         delay(20);
232     }
233
234     return sum / Measurements;
235 }
236
237 float getDistance(void) {
238
239     // Trigger a measurement
240     digitalWrite(TRIGGER, LOW);
241     delayMicroseconds(2);
242     digitalWrite(TRIGGER, HIGH);
243     delayMicroseconds(10);
244     digitalWrite(TRIGGER, LOW);
245
246     unsigned long duration = pulseIn(ECHO, HIGH, 30000); //Time for signal to come back
247
248     if (duration == 0) {
249         return -1; //kein Signal
250     }
251
252     float distance = (duration * 0.0343) / 2; // distance in cm
253
254     if (distance > 300) { // HC-SR04 sinnvoll 300 cm
255         return -1;
256     }
```

```
257
258     return distance;
259
260 }
261
262 void setup() {
263     Serial.begin(115200);
264
265     // initialize Objects Moisture Sensors
266     for (int i = 0; i < 6; i++) {
267         sensors[i].initialize(sensorPins[i]);
268     }
269
270     //***** Pin Modes *****
271     pinMode (TRIGGER, OUTPUT); // Trigger of ultrasonic sensor
272     pinMode (ECHO, INPUT); // Echo of ultrasonic sensor
273     digitalWrite(TRIGGER, LOW); //Disable trigger
274
275
276     initWiFi();
277     CSV.initSDCard(CS_SD);
278
279
280     //***** Initialize webserver *****
281     server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
282         request->send(SD, "/index.html", "text/html");
283     });
284
285     server.serveStatic("/", SD, "/");
286     server.begin();
287
288
289     //***** Local time synchronization *****
290     // get the local time from time server above
291     struct tm timeinfo;
292     configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
293
294     if(!getLocalTime(&timeinfo))
295     {
296         Serial.println("Failed to obtain time"); // Error handling
297         return;
298     }
299     else
300     {
301         Serial.println(&timeinfo, "%B-%d-%Y %H:%M:%S \n\n"); // Print timestamp
302     }
303
304
305
306     //***** SD Card *****
307     // open csv on the SD card
308     File data = SD.open("/data.csv", FILE_WRITE);
309     if (!data) {
310         Serial.println("Unable to open file"); // Error handling
```

```
311     } else {
312         data.close();
313     }
314
315     if (!bmp.begin()) {
316         Serial.println("Kein BMP085/BMP180 gefunden! Verkabelung prüfen.");
317         while (1);
318     }
319
320     // ***** BMP *****
321
322     if (event.pressure) {
323         Serial.print("Druck: ");
324         Serial.print(event.pressure);
325         Serial.println(" hPa");
326
327         /* Temperatur holen */
328         float temperature;
329         bmp.getTemperature(&temperature);
330
331         Serial.print("Temperatur: ");
332         Serial.print(temperature);
333         Serial.println(" °C");
334
335         /* Höhe berechnen */
336         float seaLevelPressure = 1013.25; // hPa (Standardwert)
337         float altitude = bmp.pressureToAltitude(seaLevelPressure, event.pressure);
338
339         Serial.print("Höhe: ");
340         Serial.print(altitude);
341         Serial.println(" m");
342
343         Serial.println("");
344     } else {
345         Serial.println("Fehler beim Auslesen des Drucks");
346     }
347
348 }
349 /*
350 void loop() {
351     CSV.appendSensorData();
352     delay(4000);
353     Serial.println("next data set");
354 }*/
355
356 void loop() {
357     bmp.getEvent(&event);
358
359     /* Druck ausgeben */
360     if (event.pressure) {
361         Serial.print("Druck: ");
362         Serial.print(event.pressure);
363         Serial.println(" hPa");
364     }
```

```
365     /* Temperatur holen */
366     float temperature;
367     bmp.getTemperature(&temperature);
368
369     Serial.print("Temperatur: ");
370     Serial.print(temperature);
371     Serial.println(" °C");
372
373     /* Höhe berechnen */
374     float seaLevelPressure = 1013.25; // hPa (Standardwert)
375     float altitude = bmp.pressureToAltitude(seaLevelPressure, event.pressure);
376
377     Serial.print("Hoehe: ");
378     Serial.print(altitude);
379     Serial.println(" m");
380
381     Serial.println("");
382 } else {
383     Serial.println("Fehler beim Auslesen des Drucks");
384 }
385
386 delay(2000);
387 }
```